INNOVACODE SOFTWARE SOLUTIONS · PROYECTO FONELLA

# Guía de Arquitectura de Software: Estructura Modular por Dominio

Manual de alineación técnica para Sergio Rodríguez (Full Stack Dev) y Diego Barrera (Product Owner) — Fundamentos, organización de carpetas, separación de responsabilidades y flujo de datos en Backend (NestJS) y Frontend (Next.js 14).

**¿Por qué este documento?** Para que todo el equipo trabaje bajo los mismos estándares de ingeniería de software. Adoptamos una **Arquitectura Modular por Dominio (Feature-Driven / Domain-Driven)** en la que el código se organiza alrededor de las *capacidades del negocio clínico de Fonella* y no por carpetas técnicas genéricas. Esto garantiza que Sergio (Carril 2), Diego (Carril 3) y Jhoan (Carril 1) desarrollen en paralelo sin colisiones en Git ni acoplamiento indeseado.

## 1. ¿Qué es la Arquitectura Modular por Dominio y por qué la elegimos?

En proyectos tradicionales se suele agrupar el código por tipo técnico (todos los controladores juntos, todos los servicios juntos). En Fonella rechazamos ese modelo porque genera archivos gigantescos y alta fricción en Git. Comparamos ambos enfoques:

| Criterio | Arquitectura Tradicional por Capas | Arquitectura Modular por Dominio (Fonella) |
|---|---|---|
| **Organización** | controllers/, services/, models/ con todos los archivos mezclados. | **Por Módulos Clínicos:** Cada dominio (Auth, Citas, Inventario) es una carpeta autocontenida. |
| **Colaboración Git** | Conflictos constantes porque varios integrantes tocan el mismo archivo de rutas o servicios. | **0 Conflictos:** Sergio trabaja en citas/ e inventario/ sin tocar los archivos de Jhoan o Diego. |
| **Mantenibilidad** | Para modificar una función hay que abrir 5 carpetas distintas repartidas en el proyecto. | **Alta Cohesión:** DTOs, Controlador, Servicio y Pruebas del módulo viven juntos en la misma carpeta. |

## 2. Backend (NestJS 10 + Prisma ORM + PostgreSQL 16)

El backend se encuentra en fonella-backend/ y expone una API REST profesional bajo el prefijo global /api/v1. Su estructura interna se divide en **Módulos de Dominio** y **Capa Transversal (Common)**:

```
fonella-backend/src/
■■■ common/                    # Capa transversal compartida
■   ■■■ decorators/            # @Roles(), @CurrentUser()
■   ■■■ filters/               # Manejador global de excepciones HTTP
■   ■■■ guards/                # JwtAuthGuard, RolesGuard (RBAC)
■■■ config/                    # Variables de entorno y validación
■■■ modules/                   # MÓDULOS DE DOMINIO INDEPENDIENTES
■   ■■■ auth/                  # HU-01: Login, Bcrypt, Cookie JWT, /me, Logout
■   ■■■ usuarios/              # HU-02: Gestión de personal (Bárbara / Secretaria)
■   ■■■ citas/                 # HU-06, HU-10, HU-11: Catálogo servicios y agenda 45m
■   ■■■ historias/             # HU-15, HU-16: Anamnesis y notas de evolución
■   ■■■ inventario/            # HU-07, HU-20: Catálogo juguetes y stock
■   ■■■ notificaciones/        # HU-22: Protocolo recordatorios WhatsApp
■■■ prisma/                    # Servicio singleton de Prisma Client
■■■ app.module.ts              # Módulo raíz que ensambla los submódulos
■■■ main.ts                    # Entrypoint (CORS, CookieParser, ValidationPipe)
```

### Anatomía Estándar de Cada Módulo en Backend:

Cada carpeta dentro de src/modules/{nombre} debe seguir esta estricta separación de responsabilidades:

• ***.controller.ts:** Puerta de entrada HTTP. Define rutas, verbos (GET, POST, PATCH), decoradores Swagger y guardianes de seguridad (@UseGuards(JwtAuthGuard, RolesGuard)). *No contiene lógica de negocio*.

• ***.service.ts:** El cerebro del módulo. Ejecuta las reglas clínicas, validaciones de unicidad, cálculos y consultas a PostgreSQL a través de PrismaService.

• **dto/*.dto.ts:** Contratos de entrada y salida con validaciones estrictas usando class-validator (ej. @IsEmail(), @MinLength(8)).

• ***.module.ts:** Declara qué controladores y servicios pertenecen al módulo y qué exporta para otros.

## 3. Frontend (Next.js 14 App Router + Tailwind CSS)

El frontend en fonella-frontend/ utiliza la arquitectura de **Grupos de Rutas (Route Groups)** de Next.js 14. Las carpetas entre paréntesis (nombre) organizan los contextos sin alterar la URL visible en el navegador:

```
fonella-frontend/src/
■■■ app/
■   ■■■ (auth)/                # CONTEXTO AUTENTICACIÓN
■   ■   ■■■ login/page.tsx     # URL: /login (Tarjeta centrada y fondo clínico)
■   ■■■ (panel)/               # CONTEXTO ADMINISTRATIVO PRIVADO
■   ■   ■■■ layout.tsx         # Shell privado (Sidebar #136A7B, perfil Dra. Bárbara)
■   ■   ■■■ personal/page.tsx  # URL: /personal (HU-02: Tabla y alta de secretaria)
■   ■   ■■■ agenda/page.tsx    # URL: /agenda (Sprint 2: Calendario 45 min)
■   ■   ■■■ historias/page.tsx # URL: /historias (Sprint 2: Expedientes clínicos)
■   ■■■ (publico)/             # CONTEXTO PORTAL WEB PÚBLICO
■   ■   ■■■ layout.tsx         # Navbar público y footer institucional
■   ■   ■■■ page.tsx           # URL: / (HU-05: Hero, RETHUS, biografía y servicios)
■   ■   ■■■ catalogo-juguetes/ # URL: /catalogo-juguetes (HU-07: Vitrina didáctica)
■   ■   ■■■ solicitar-cita/    # URL: /solicitar-cita (HU-09: Formulario asistido)
■   ■■■ globals.css            # Directivas Tailwind y estilos base
■   ■■■ layout.tsx             # Root Layout HTML con fuentes y metadatos
■■■ components/                # Componentes UI reutilizables (Botones, Modales, Tarjetas)
■■■ services/                  # Capa de consumo API (api.ts con credenciales y fetch)
■■■ utils/                     # Funciones auxiliares puras (whatsapp-link.ts)
```

### La Capa Desacoplada de Servicios (services/api.ts):

**Regla de oro:** Ningún componente de React debe hacer llamadas directas con fetch('http://localhost:4000/...') esparcidas en el código. Todas las peticiones deben pasar por apiService en src/services/api.ts:

1. **Manejo de Cookies Automático:** Incluye credentials: 'include' para que la cookie segura fonella_token viaje automáticamente.
2. **Manejo Centralizado de Errores:** Si el backend responde 401/403/409, apiService captura el mensaje legible del DTO y lo entrega limpio a la interfaz.
3. **Tipado TypeScript Estricto:** Todo endpoint retorna interfaces de TypeScript (Usuario, Servicio, Juguete) para autocompletado y cero errores en compilación.

## 4. Hoja de Ruta Específica para Sergio (Carril 2) y Diego (Carril 3)

Para que sepan con precisión quirúrgica qué archivos crear y dónde trabajar en el Sprint 1:

| Integrante / Carril | Archivos de Backend | Archivos de Frontend |
|---|---|---|
| **Sergio Rodríguez**<br>Carril 2 (8 pts)<br>HU-05, HU-06, HU-07 | • modules/citas/citas.controller.ts (Endpoint público GET /servicios)<br>• modules/inventario/inventario.controller.ts (Endpoint público GET /catalogo)<br>• *Consultar datos mediante PrismaService sin requerir token JWT*. | • app/(publico)/page.tsx (Sección Hero, RETHUS y servicios)<br>• app/(publico)/catalogo-juguetes/page.tsx (Vitrina con filtro de categorías y stock)<br>• services/api.ts (Añadir métodos listarServicios() y listarCatalogoJuguetes()). |
| **Diego Barrera**<br>Carril 3 (6 pts)<br>HU-08, HU-09 | • *No requiere endpoints backend nuevos en Sprint 1* (conversión directa cliente → WhatsApp Business).<br>• Preparar base de datos para registrar trazabilidad de clics en Sprint 2. | • utils/whatsapp-link.ts (Helper de sanitización de teléfono y codificación URI)<br>• components/ui/BotonFlotanteWhatsApp.tsx (Botón flotante fijo en esquina inferior)<br>• app/(publico)/solicitar-cita/page.tsx (Formulario asistido menor/adulto con redirección a wa.me). |

## 5. Las 6 Reglas de Oro de Arquitectura en InnovaCode

**1. Prohibido mezclar dominios:** El módulo de citas no debe importar modelos ni modificar tablas de inventario directamente. Cada módulo se comunica por su interfaz pública.

**2. Controladores limpios, Servicios inteligentes:** El controlador solo valida la petición y llama al servicio. Toda la lógica de negocio vive en el *.service.ts.

**3. Cero llamadas HTTP sueltas en React:** Todo componente de Next.js consume los datos a través de apiService. Queda prohibido escribir fetch() directo en las páginas.

**4. Seguridad por defecto:** Toda ruta del panel administrativo lleva @UseGuards(JwtAuthGuard, RolesGuard). Queda prohibido dejar endpoints administrativos desprotegidos.

**5. Identidad Visual Estricta:** Todo componente visual debe usar exclusivamente las clases de Tailwind configuradas con la paleta de Fonella (Teal #136A7B, Turquesa #0AA896, Terracota #DF9B78, Arena #F5ECDD).

**6. Integración Continua sin Romper Develop:** Antes de solicitar un PR, ejecutar npm run build en backend y frontend. Si falla la compilación de TypeScript, no se aprueba el merge.

### Control de Versiones del Documento

| Versión | Fecha | Autor | Cambios Principales |
|---|---|---|---|
| v1.0 Oficial | 03/Nov/2026 | Jhoan Borja (Scrum Master & Software Architect) | Definición de arquitectura modular por dominio, mapa de carpetas y guía técnica para Sprint 1. |